

Reviewer Pack — Minimal Index of Algebraic Coadjointness and Communication C...

Entry 0618ff011015 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 10:59:28 UTC

Minimal Index of Algebraic Coadjointness and Communication Complexity Rank

Entry ID: 0618ff011015 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 10:59:28 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Coadjoint Representations) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For all boolean functions f with communication complexity rank $r(f)$, the minimal index of coadjointness for the Lie algebra associated with f is $O(r(f)^3)$.

Rationale (proposer's reasoning):

The index of coadjointness in Lie theory measures the complexity of coadjoint orbits, which are related to the structure of representations. This conjecture suggests that such a structural invariant could capture aspects of communication complexity, potentially revealing new connections between algebraic structures and complexity measures.

Taxonomy category: lie_theory_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3c27559a757268d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal index of coadjointness for the Lie algebra associated with a boolean function is within $O(r(f)^3)$, where $r(f)$ is the communication complexity rank of the function.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Lie Theory" AND "coadjoint representations" AND "communication complexity" AND "matrix rank" - "minimal index" AND "algebraic coadjointness" AND "communication complexity rank" - "boolean functions" AND "communication complexity" AND "Lie algebra" AND "minimal index of coadjointness"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    rank += 1
        return rank

    def construct_lie_algebra(f):
        n = len(f)
        lie_algebra = []
        for i in range(n):
            for j in range(i+1, n):
                if f[i] == f[j]:
                    lie_algebra.append((i, j))
        return lie_algebra

    def minimal_index_of_coadjointness(lie_algebra):
        n = len(lie_algebra)
        index = 0
        for i in range(n):
            for j in range(i+1, n):
                if (lie_algebra[i][0], lie_algebra[j][0]) not in lie_algebra and (lie_algebra[i][1],
```

```

        index += 1
    return index

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
    return cov / (std_x * std_y)

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    r_f = communication_complexity_rank(f)
    lie_algebra = construct_lie_algebra(f)
    index_coadjointness = minimal_index_of_coadjointness(lie_algebra)
    results.append((r_f, index_coadjointness))

if not results:
    return {
        "metric_name": "Pearson correlation",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

r_f_values, index_coadjointness_values = zip(*results)
correlation = pearson_correlation(r_f_values, index_coadjointness_values)

return {
    "metric_name": "Pearson correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(len(f) for f in [generate_boolean_function(n) for n in [5, 10, 15, 20, 30, 40]]),
    "conjecture_holds": abs(correlation) >= 0.9,
    "counterexample": "" if abs(correlation) >= 0.9 else "Pearson correlation < 0.9"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results))

```

```

support_fraction = 1.0
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    mean_value = sum(result["metric_value"] for result in results if result["conjecture_holds"]) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results if result["conjecture_holds"]) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

print(f"RESULT: {'SUPPORTED' if all(result['conjecture_holds'] for result in results) else 'FAILED'}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition and therefore cannot be confirmed as SUPPORTED. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14146
2	propose	ollama_remote	glm4:latest	0	0	13543
3	preregistration	ollama_remote	glm4:latest	0	0	9947
4	novelty	ollama_remote	glm4:latest	0	0	9350
5	novelty	ollama_remote	glm4:latest	0	0	9082
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29836
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12809
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9818
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13908
10	judge	ollama_remote	glm4:latest	0	0	16446

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138885 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in <research/audit/0618ff011015.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0618ff011015.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0618ff011015.tar.gz` (if generated)